



Working With Data.

Yesterday you organized widgets. Today we fill them with real data — CSV files, spreadsheets, tables. Your apps start remembering things.

PROGRAM

Python Internship

LED BY

Zlaark

SESSION

6 of 24

How we'll spend our 90 minutes today.

Today is the densest day of Phase 2 — six pandas concepts, one cheatsheet, and a dashboard exercise that combines everything you have learned so far.

DURATION
90 min

00:00 – 00:10



Day 5 Recap & Q&A

Layout primitives, session_state, the personal info form. Quick check.

00:10 – 00:25



DataFrames + Loading Data

What a DataFrame is, how to load a CSV, the three `read_*` functions you will actually use.

00:25 – 00:40



Inspecting + Filtering + Sorting

`head` / `info` / `describe`, boolean masks, `sort_values`.
The 80% you will do every day.

00:40 – 00:55



loc vs iloc + New Columns

Label-based vs position-based indexing. Adding and modifying columns.

00:55 – 01:10



pandas Cheatsheet

The 10 commands you'll use 90% of the time. A printable reference card worth bookmarking.

01:10 – 01:30



Exercise: Student Dashboard

Build a Streamlit app that loads a CSV and filters by branch/city. Combines Day 5 sidebar + Day 6 pandas.

Yesterday you built a form. Today we feed it data.

Quick check that Day 5's form app runs. If it doesn't, we fix it now — today's exercise builds on the sidebar pattern.

WHAT WE COVERED YESTERDAY DAY 05

- ✓ **Widget keys** — the habit that pays off in Phase 4
- ✓ **The 5 layout primitives** — `columns`, `sidebar`, `tabs`, `expander`, `container` / `empty`
- ✓ **`session_state`** — making variables persist across the run-loop
- ✓ **Layout patterns cheatsheet** — when to use which primitive
- ✓ **Built a personal info form** with `st.form` — the Phase-4 data-collection pattern

CHECK YOUR LAPTOP NOW VERIFY

```
# You should have Day 5's form files:
$ ls day5/
10_form_solution.py
11_form_extended.py
...

# Run the form — should open in browser:
$ streamlit run 10_form_solution.py
# → Browser opens at localhost:8501
# → Fill the form, click Submit
# → Success message + summary appears
```

⚠ **IF THE FORM DOESN'T RUN** — raise your hand. Common issue: forgot to activate `.venv` first. Run `source .venv/bin/activate` (macOS) or `.venv\Scripts\Activate.ps1` (Windows), then retry.

pandas is a spreadsheet you control with code.

You already know spreadsheets — rows, columns, filters, sorts. pandas gives you the same operations, but as Python code. Repeatable, scriptable, and able to handle files Excel chokes on.

IN EXCEL

WHAT YOU KNOW



Rows and columns

A grid of data. Each column has a name, each row has a position.



Filters

Show only rows where column X meets a condition.



Sorts

Reorder rows by one or more columns, ascending or descending.



Formulas

SUM, AVERAGE, COUNT — aggregate a column or a selection.



Pivot tables

Group by one column, aggregate another. The most powerful Excel feature.

Powerful, but every action is a click. Not repeatable. Not scriptable.

IN PANDAS

THE CODE VERSION



DataFrame

A table. Rows have an index, columns have names. The central pandas object.



`df[df["age"] > 20]`

Boolean mask filtering. One line of code, repeatable.



`df.sort_values("marks")`

Sort by a column. Chain multiple sorts.



`df["marks"].mean()`

Aggregate a column. Same idea, code syntax.



`df.groupby("city")["marks"].mean()`

Group by + aggregate. The pivot table, as one line.

Same operations. Code instead of clicks. Scriptable, repeatable, scales to millions of rows.

KEY IDEA

If you can do it in Excel, you can do it in pandas. The mental model is the same — the syntax is **Python**.

• CONCEPT • THE DATAFRAME

A DataFrame is a table. Rows have an index, columns have names.

That's it. Every pandas operation is about selecting rows, selecting columns, or transforming them. Once you internalise this mental model, the API becomes obvious.

A DATAFRAME — STUDENTS.CSV

COLUMNS ↓ NAMES					
index	name	age	city	branch	marks
0	Aarav	20	Amritsar	BCA	85
1	Priya	21	Jalandhar	BCA	92
2	Rahul	22	Amritsar	BSc-IT	78
3	Meera	20	Ludhiana	BCA	88

INDEX →

ROW ← D COLUMN ↑ DF["MARKS"]

THE MENTAL MODEL

1 A DataFrame is 2D

Rows + columns. Like a spreadsheet. Like a SQL table. Like a CSV file loaded into memory.

2 A Series is 1D

A single column (or a single row). Every column in a DataFrame is a Series. Most operations work on both.

3 The index is NOT a column

The leftmost numbers (0,1,2,3) are the index — row labels, not data. Don't confuse them with the first data column.

4 dtypes matter

Each column has a dtype: `int64`, `float64`, `object` (strings), `bool`, `datetime64`. Mismatched dtypes cause silent bugs. Check with `df.dtypes`.

KEY IDEA Selecting rows, selecting columns, transforming them. That's **90% of pandas**. The other 10% is just syntax.

• CONCEPT • LOADING DATA

Three loaders. `read_csv` covers 90% of what you will ever do.

pandas can read CSV, Excel, JSON, SQL, HTML tables, Parquet, and a dozen other formats. In this program, you will use three. Master those three.



`pd.read_csv(filepath)`

CSV files. The most common format. Comma-separated, plain text. What you will use 90% of the time.

BEST FOR Tabular data exported from any tool. Logs, exports, datasets.



`pd.read_excel(filepath)`

Excel files (`.xlsx` , `.xls`). Requires the `openpyxl` package — `pip install openpyxl` .

BEST FOR When the source is an Excel file with formatting, multiple sheets, or formulas.



`pd.read_json(filepath)`

JSON files. Common for API responses. Each row becomes a dict, the list of dicts becomes a DataFrame.

BEST FOR Data from web APIs (Day 11), NoSQL databases, config files.

loading_demo.py

```
import pandas as pd

# === Load a CSV ===
df = pd.read_csv("students.csv")
# → returns a DataFrame

# === Common useful arguments ===
df = pd.read_csv(
    "students.csv",
    sep=";",          # delimiter (default comma)
    header=0,         # row number to use as column names
    index_col=0,       # column to use as row index
    nrows=100,         # only load first 100 rows (big files)
    encoding="utf-8",  # for files with non-ASCII chars
)

# === Loading from a URL (Day 11 preview) ===
url = "https://example.com/data.csv"
df = pd.read_csv(url) # works the same way!

# === Quick check it loaded ===
print(df.shape)      # (rows, cols)
print(df.head())     # first 5 rows
```

TIP If `read_csv` fails, check: (1) is the file in the same folder? (2) is the path correct? (3) is the encoding `utf-8` ? 90% of loading errors are one of these three.

• CONCEPT • INSPECTING DATA

Three commands. Run them every time you load a DataFrame.

Before you do anything else, run these three. They tell you the shape, the structure, and the statistical summary. Five seconds of inspection saves an hour of debugging.

`df.head(n=5)`



First n rows. Confirms the data loaded correctly and the columns look right.

```
df.head()      # default 5 rows
df.head(10)    # first 10 rows
```

name	age	city	branch	marks
Aarav	20	Amritsar	BCA	85
Priya	21	Jalandhar	BCA	92
Rahul	22	Amritsar	BSc-IT	78
Meera	20	Ludhiana	BCA	88
Sanjana	21	Amritsar	BCA	91

`df.info()`



Structure summary. Column names, dtypes, non-null counts, memory usage. The most overlooked command.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 28 entries, 0 to 27
Data columns (total 5 columns):
#   Column  Non-Null Count  Dtype
---  -
0   name    28 non-null      object
1   age     28 non-null      int64
2   city    28 non-null      object
3   branch  28 non-null      object
```

`df.describe()`



Statistical summary of numeric columns. Count, mean, std, min, max, quartiles.

	age	marks
count	28.0	28.0
mean	20.8	84.7
std	1.0	6.4
min	19.0	72.0
25%	20.0	80.0
50%	21.0	85.0
75%	21.0	90.0

BONUS

`df.shape` returns (rows, cols) as a tuple. `df.columns` returns the column names. `df.dtypes` returns the dtype of each column. Run these too.

Filtering = create a **boolean mask**, then use it to select rows.

This is the single most important pandas pattern. Once you get boolean masks, every filter becomes obvious.

THE MENTAL MODEL

STEP 1 CREATE A MASK (SERIES OF TRUE/FALSE, ONE PER ROW)

`df["age"] > 20`

→ [False, True, True, True, False, ...]



STEP 2 USE THE MASK TO SELECT ROWS

`df[mask]`

→ only rows where True

- 1 **The mask is a Series** — Same length as the DataFrame. Each value is True or False.
- 2 **`df[mask]` returns rows where True** — A filtered DataFrame. Original is unchanged.
- 3 **Combine masks with & (and), | (or), ~ (not)** — Wrap each in parentheses: `df[(df["age"] > 20) & (df["city"] == "Amritsar")]`.

filtering_demo.py

PYTHON

```
import pandas as pd
df = pd.read_csv("students.csv")

# === Step 1: Create a mask ===
# A mask is a Series of True/False, one per row.
mask = df["age"] > 20
# → [False, True, True, True, False, ...]

# === Step 2: Use the mask to select rows ===
older = df[mask]
# → only rows where mask is True

# Or as a one-liner (common in real code):
older = df[df["age"] > 20]

# The original df is UNCHANGED.
# df[mask] returns a new filtered DataFrame.
# Next slide: combining multiple conditions.
```

KEY IDEA Every filter is two steps: **create a mask** (a True/False Series), then **use it to select rows** (`df[mask]`). Master this pattern, and every filter becomes obvious. Next slide: combining multiple conditions.

Combining conditions: AND, OR, contains, isin.

One condition is rarely enough. Real filters combine multiple conditions — and pandas has specific syntax for each pattern.

filtering_advanced.py

PYTHON

```
import pandas as pd
df = pd.read_csv("students.csv")

# === 1. Multiple conditions (AND) ===
# Both must be True. Use & (bitwise and).
# WRAP EACH CONDITION IN PARENTHESES.
amritsar_bca = df[
    (df["city"] == "Amritsar") &
    (df["branch"] == "BCA")
]

# === 2. Multiple conditions (OR) ===
# Either can be True. Use | (bitwise or).
young_or_old = df[
    (df["age"] < 20) |
    (df["age"] > 22)
]

# === 3. String contains ===
```

FOUR PATTERNS

- &** **AND** — **&** — Both conditions must be True. Wrap each in parens: `(df["x"] > 5) & (df["y"] < 10)`.
- |** **OR** — **|** — Either condition can be True. Same paren rule: `(df["x"] > 5) | (df["y"] < 10)`.
- ~** **NOT** — **~** — Invert a mask. `~(df["age"] > 20)` gives rows where age ≤ 20.
- Aa** **String contains** — **.str.contains()** — Substring match on a text column. Case-sensitive by default; add `case=False` to ignore.
- ∈** **Is in a list** — **.isin()** — Filter by a set of allowed values. Faster than chaining multiple `==` with **|**.

GOTCHA Always wrap each condition in parentheses when combining with **&** or **|**. `df[df["age"] > 20 & df["city"] == "Amritsar"]` raises a **TypeError**. Pandas operator precedence is different from Python — the parens are not optional.

sort_values — reorder rows by one or more columns.

Simple, but with two gotchas: `ascending` defaults to `True`, and `inplace` defaults to `False`. Get these wrong and you will wonder why your sort didn't work.

THE MENTAL MODEL



Single column

`df.sort_values("marks")` — sort by one column, ascending by default.



Multiple columns

`df.sort_values(["city", "marks"])` — sort by city first, then by marks within each city.



Descending

Pass `ascending=False` to flip the order.



Returns a copy

Default: returns a new DataFrame, original unchanged. Use `inplace=True` to modify the original.

sorting_demo.py

PYTHON

```
import pandas as pd
df = pd.read_csv("students.csv")

# === Sort by one column, ascending ===
sorted_df = df.sort_values("marks")
# lowest marks first

# === Sort by one column, descending ===
top_first = df.sort_values("marks", ascending=False)
# highest marks first

# === Sort by multiple columns ===
# Sort by city (A-Z), then by marks (high-low) within each city
city_then_marks = df.sort_values(
    ["city", "marks"],
    ascending=[True, False]
)

# === Modify in place (rare – prefer the copy) ===
# df.sort_values("marks", inplace=True)
# → modifies df directly, returns None

# === Reset the index after sorting ===
sorted_df = df.sort_values("marks", ascending=False).reset_index(drop=True)
# → index becomes 0,1,2,3... again
```

PATTERN Almost always: `df.sort_values("col", ascending=False).reset_index(drop=True)`. The `reset_index` keeps the index clean after rows are reordered.

loc uses labels. iloc uses positions. They are NOT interchangeable.

This trips up every pandas beginner. `loc` looks up by row label and column name. `iloc` looks up by row number and column number. Different inputs, different outputs.

LOC • LABEL-BASED

```
df.loc[row_label, col_name]
```

- Uses the **INDEX** (row labels) and **COLUMN NAMES**
- **Inclusive** on both ends: `df.loc[0:3]` returns rows 0,1,2,3 (4 rows)

```
# Row with index label 2
df.loc[2]

# Rows 0 through 3 (INCLUSIVE – 4 rows)
df.loc[0:3]

# Specific rows + specific columns
df.loc[[0, 2, 4], ["name", "marks"]]

# Boolean mask filtering
df.loc[df["age"] > 20, ["name", "marks"]]
```

ILOC • POSITION-BASED

```
df.iloc[row_position, col_position]
```

- Uses **POSITIONS** (0,1,2,3...) like a Python list
- **Exclusive** on the end: `df.iloc[0:3]` returns rows 0,1,2 (3 rows)

```
# Row at position 2 (third row)
df.iloc[2]

# Rows 0 through 2 (EXCLUSIVE – 3 rows)
df.iloc[0:3]

# First 3 rows, first 2 columns
df.iloc[0:3, 0:2]

# Last row
df.iloc[-1]
```

GOTCHA When the index is default (0,1,2,3...), `df.loc[2]` and `df.iloc[2]` return the same row. But `df.loc[0:3]` returns **4 rows**, `df.iloc[0:3]` returns **3 rows**. The slicing is different. Always check which one you meant.

New columns = old columns + an operation. Same syntax as variables.

Creating a new column is just `df["new"] = ...`. The right side can be a constant, a calculation on other columns, or a function applied row-by-row.

THREE PATTERNS

1

Constant value

```
df["status"] = "active"
```



every row gets the same value.

2

Column arithmetic

```
df["total"] = df["marks"] + df["bonus"]
```



element-wise operation on two columns.

3

Row-wise transform

```
df["grade"] = df["marks"].apply(to_grade)
```



apply a function to each value in a column.

columns_demo.py

PYTHON

```
import pandas as pd
df = pd.read_csv("students.csv")

# === 1. Add a constant column ===
df["school"] = "Hindu College Amritsar"

# === 2. Column arithmetic ===
# Convert marks (out of 100) to percentage
df["percentage"] = df["marks"] / 100 * 100

# === 3. Apply a function ===
def to_grade(marks: int) -> str:
    if marks >= 90: return "A"
    if marks >= 80: return "B"
    if marks >= 70: return "C"
    return "F"

df["grade"] = df["marks"].apply(to_grade)

# === 4. Modify an existing column ===
df["city"] = df["city"].str.upper()
# -> "amritsar" becomes "AMRITSAR"

# === 5. Conditional column (np.where) ===
import numpy as np
df["passed"] = np.where(df["marks"] >= 40, "Yes", "No")

# === 6. Drop a column ===
df = df.drop(columns=["school"])
# or: df.drop("school", axis=1, inplace=True)
```

PATTERN `df["new_col"] = ...` is how you create or overwrite. The right side can be anything that produces a Series of the right length: a constant, column math, `.apply()`, `np.where()`, or another column.

Bookmark this slide. These 10 commands are 90% of pandas.

You've seen most of these today. Here they are in one place, with a one-line example each. Print this. Tape it to your laptop.

01

pd.read_csv(path) Load a CSV file into a DataFrame.

```
df = pd.read_csv("students.csv")
```

02

df.head(n) First n rows. Default 5. Quick sanity check.

```
df.head(10)
```

03

df.info() Structure summary. Columns, dtypes, non-null counts.

```
df.info()
```

04

df.describe() Statistical summary of numeric columns.

```
df.describe()
```

05

df["col"] Select one column. Returns a Series.

```
names = df["name"]
```

06

df[mask] Filter rows with a boolean mask.

```
df[df["age"] > 20]
```

07

df.sort_values("col") Sort by a column. ascending=True by default.

```
df.sort_values("marks", ascending=False)
```

08

df.loc[row, col] Label-based selection. Rows + columns by name.

```
df.loc[0:3, ["name", "marks"]]
```

09

df["new"] = ... Add or modify a column.

```
df["grade"] = df["marks"].apply(to_grade)
```

10

df.groupby("col") Group rows by a column. Chain with .mean(), .sum(), .count().

```
df.groupby("city")["marks"].mean()
```

PATTERN Load with `read_csv`, inspect with `head/info/describe`, filter with `df[mask]`, sort with `sort_values`, select with `loc`, transform with `df["new"] = ...`. That's the whole workflow.

03 / 03

Build a student dashboard. Sidebar + pandas.

Today's exercise combines Day 5's sidebar pattern with Day 6's pandas. Load a CSV, filter by branch and city, display the results. The dashboard pattern every group will reuse in Phase 4.

1 BRIEF • 5 MIN

2 BUILD • 15 MIN

3 EXTEND • 5 MIN

Load `students.csv` . Filter by branch and city. Display the result.

A real dashboard pattern. Controls in the sidebar (Day 5), data filtering with pandas (Day 6), results in the main area. Every group project will have some version of this.

THE BRIEF

TASK

Build a Streamlit app called `dashboard.py` that loads `students.csv` , displays the full table, and lets the user filter by branch and city using sidebar widgets.

REQUIREMENTS

- ✓ Load the CSV with `pd.read_csv`
- ✓ Show the full DataFrame using `st.dataframe`
- ✓ Sidebar: a selectbox for branch (All + unique branches)
- ✓ Sidebar: a selectbox for city (All + unique cities)
- ✓ Filter the DataFrame based on selections
- ✓ Show the filtered DataFrame below the full one

BY THE END

Every student has a working dashboard that filters a real CSV by two columns. The pattern groups will reuse in Phase 4.

SAMPLE DATA

CSV

`students.csv`

```
name,age,city,branch,marks
Aarav,20,Amritsar,BCA,85
Priya,21,Jalandhar,BCA,92
Rahul,22,Amritsar,BSc-IT,78
Meera,20,Ludhiana,BCA,88
Sanjana,21,Amritsar,BCA,91
Vikram,23,Jalandhar,BSc-IT,75
Anjali,20,Amritsar,BCA,89
Karan,22,Ludhiana,BCA,82 ... (28 rows total)
```

STARTING SCAFFOLD

TODO

```
# dashboard.py - starting scaffold
import streamlit as st
import pandas as pd
st.title("Student Dashboard")
# 1. Load the data
df = pd.read_csv("students.csv")
# 2. Sidebar controls
with st.sidebar:
    st.write("**Filters**")
    # TODO: branch selectbox
    # options = ["All"] + df["branch"].unique().tolist()
    # branch = st.selectbox("Branch", options)
    # TODO: city selectbox
    # city = st.selectbox("City", options)
# 3. Show full data
st.write("**All students**")
st.dataframe(df)
# 4. Filter (TODO)
# filtered = df
# if branch != "All":
#     filtered = filtered[filtered["branch"] == branch]
# if city != "All":
#     filtered = filtered[filtered["city"] == city]
# 5. Show filtered (TODO)
# st.write(f"**Filtered: {len(filtered)} students**")
# st.dataframe(filtered)
```

RUN IT `streamlit run dashboard.py` . Pick a branch + city in the sidebar. The filtered table updates live.

One way to solve it. There are others — this is just clean.

Walk through this line by line. Then go back to your own version and compare.

dashboard.py

SOLUTION

```
import streamlit as st
import pandas as pd

1 st.title("Student Dashboard")

2 df = pd.read_csv("students.csv")

3 with st.sidebar:
    st.write("**Filters**")
    branches = ["All"] + df["branch"].unique().tolist()
    cities = ["All"] + df["city"].unique().tolist()
    branch = st.selectbox("Branch", branches, key="filter_branch")
    city = st.selectbox("City", cities, key="filter_city")

4 st.write("**All students**")
st.dataframe(df)
st.write(f"Total: {len(df)} students")

5 filtered = df
if branch != "All":
    filtered = filtered[filtered["branch"] == branch]
if city != "All":
    filtered = filtered[filtered["city"] == city]

6 st.write(f"**Filtered: {len(filtered)} students**")
st.dataframe(filtered)
if len(filtered) > 0:
    st.write(f"Average marks: {filtered[\"marks\"].mean():.1f}")
```

LINE BY LINE

6 STEPS

- 1 **Title**
Standard `st.title`. The page heading.
- 2 **Load the data**
`pd.read_csv` loads the CSV into a DataFrame. One line.
- 3 **Sidebar controls**
Build option lists: `["All"] + unique values`. The "All" option is the no-filter default. Both widgets keyed.
- 4 **Show full data**
`st.dataframe` renders an interactive table. Below it, a count of total rows.
- 5 **Filter**
Start with the full df, then apply each filter only if the user picked something other than "All". The universal dashboard pattern.
- 6 **Show filtered + stats**
`st.dataframe` again for the filtered result. Bonus: average marks of the filtered set, formatted to 1 decimal.

OTHER SOLUTIONS EXIST — You could use `st.data_editor` for editable tables, `st.metric` for the count, or `groupby` for per-branch averages. All valid. This version is the most readable for a beginner.

Finished early? Add stats. Or peek at tomorrow.

Three extension ideas for fast students. Plus a preview of Day 7 — when your data gets visual.

EXTENSION IDEAS

FOR FAST STUDENTS

1 Add a marks-range slider

In the sidebar, add `st.slider("Marks range", 0, 100, (0, 100))` and filter: `filtered = filtered[(filtered["marks"] >= low) & (filtered["marks"] <= high)]`. Now users can filter by marks range.

2 Show summary stats with `st.metric`

Below the filtered table, use `st.metric(label="Average", value=f"{filtered["marks"].mean():.1f}")` in 3 columns: average marks, highest mark, lowest mark. Looks like a real dashboard.

3 Add a search box

In the sidebar, add `st.text_input("Search by name")`. Filter: `if search: filtered = filtered[filtered["name"].str.contains(search, case=False)]`. Now users can search by name.

WHAT'S NEXT • DAY 7

CHARTS & DASHBOARDS



Line charts

Plot a column over time. `df.plot()` or `st.line_chart`.



Bar charts

Compare values across categories. `df.plot.bar()` or `st.bar_chart`.



Full dashboard

Combine sidebar + dataframe + metrics + charts. The Phase-2 capstone pattern.

TOMORROW — Your data stops being a table and starts being a picture. The moment dashboards feel real.

IF YOU'RE STUCK — Pair with a neighbour. Read the error message out loud. Raise your hand. We move at the room's pace.

Today your apps learned to remember.

Tomorrow we make that data visible — line charts, bar charts, a real dashboard. The day Phase 2 starts feeling like a product.

OPEN THE FLOOR

ASK NOW

- ✓ "Which pandas concept felt most unclear — DataFrames, filtering, loc/iloc?"
- ✓ "Did the boolean mask pattern click, or does it feel backwards?"
- ✓ "Anything from the pitfalls slide you've already hit?"

BETWEEN SESSIONS

GET UNSTUCK

- ✓ **Email** • hello@zlaark.com
For pandas questions, paste your code + the error + a sample of your CSV.
- ✓ **Web** • www.zlaark.com
Program updates and resources.
- ✓ **In-session** • **raise a hand**
No question is too small. We move at the room's pace.

WHAT'S NEXT • DAYS 7-8

PHASE 2 CONTINUES

- ✓ **Day 7** — Charts & Dashboards. Line charts, bar charts, a real dashboard.
- ✓ **Day 8** — Solo challenge. The gate before group formation on Day 15.
- ✓ **Phase 4 preview** — Every group project will use today's filter pattern.